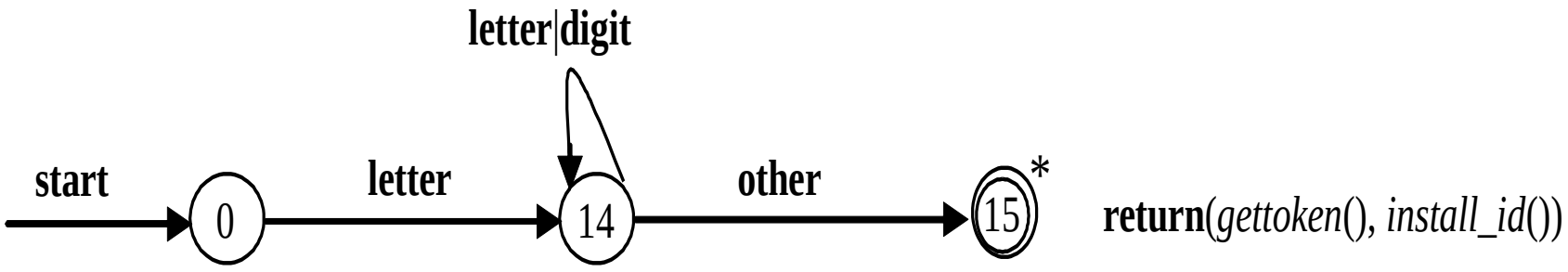
实验一—词法

单词的识别
有穷状态自动机与单词识别的关系

- 有穷状态自动机和正则文法等价，考虑到状态转换图的直观性，我们从状态转换图出发来考虑词法分析器的设计。
- 允许在状态转换图的边上标记像**digit**、**letter**这样意义明确的符号，**other**表示例外情况

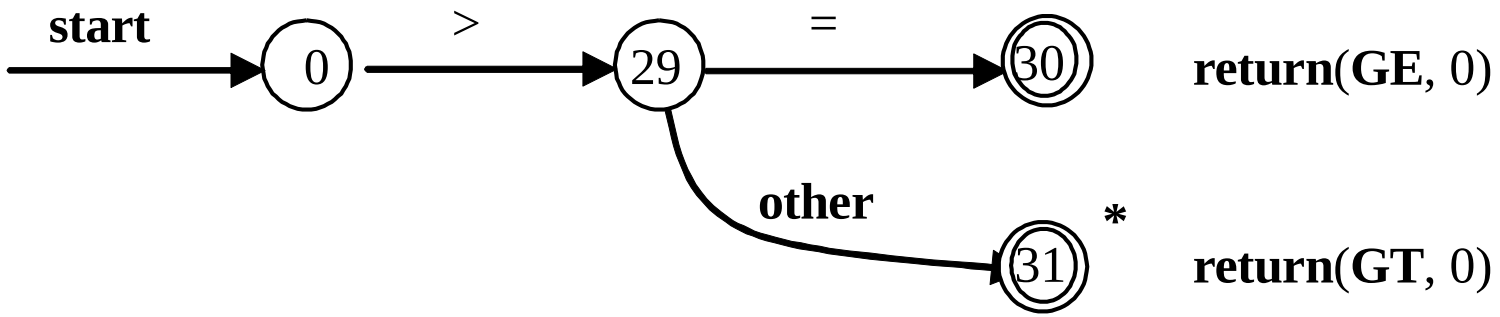


有穷状态自动机与单词识别的关系

- 单词的识别过程相当于单词的拼接过程，需要一个字符一个字符地逐步进行，而且每个单词的识别总可以在有限的步骤内完成。
- 在这个识别过程中，每读入一个字符，表示识别达到某一种状态，这样整个识别过程就可以看成是有限个状态的变换，识别系统就成了一个具有有穷个状态的变换系统，每一个状态反映的是单词的某种识别程度。

有穷状态自动机与单词识别的关系

- 考虑到在识别单词的过程中需要执行一些动作，所以将这些动作标在基本的状态转换图上。
- 如果到达终止状态，则意味着读入了一个与当前单词无关的字符，由于这个无关字符是下一个单词的开始符号，所以必须回退一个字符。
- 状态上的*表示向前指针必须回退一个字符。



例 不同进制无符号整数的识别

八进制数:　　(OCT，值)

- oct→0(0|1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)*

十进制数:　　(DEC，值)

- dec→(1|...|9)(0|...|9)* |0

十六进制数:　(HEX，值)

- hex→0x(0|1|...|9|a|...|f|A|…|F)(0|...|9|a|...|f |A|…|F)*

识别不同进制数的状态图

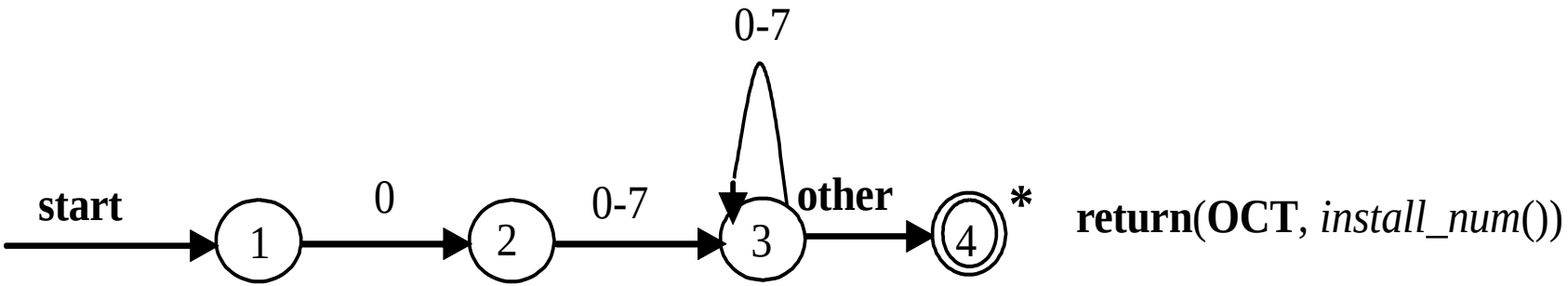


图3.11 识别八进制无符号整数的状态转换图

识别不同进制数的状态图

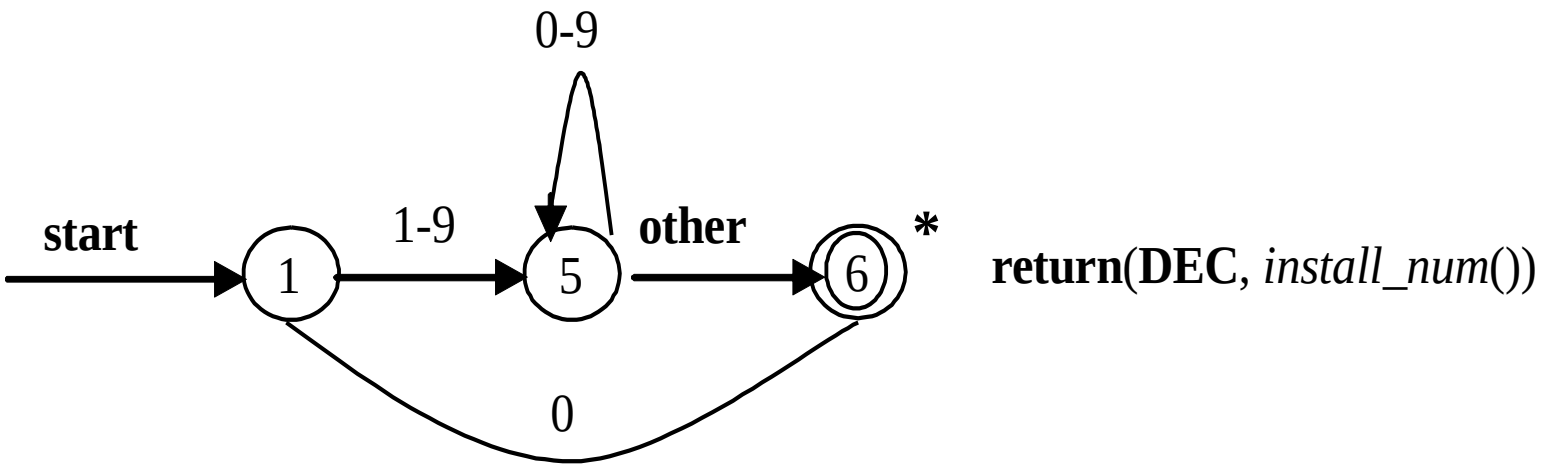


图3.12 识别十进制无符号整数的状态转换图

识别不同进制数的状态图

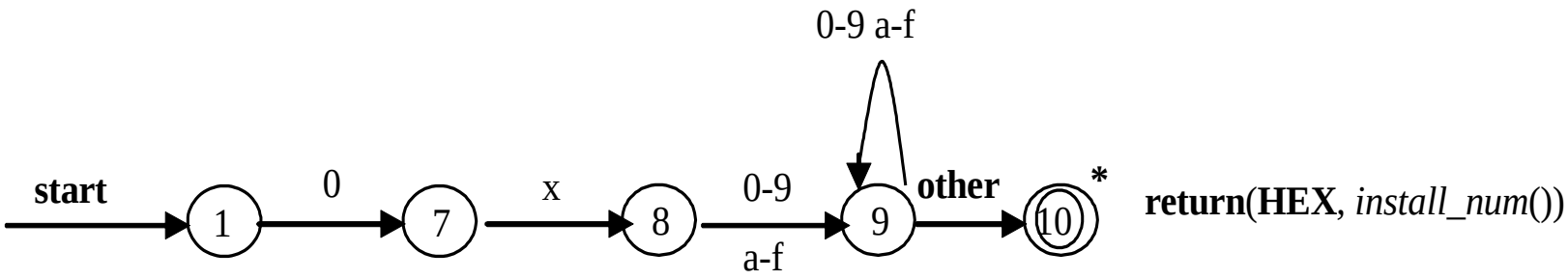


图3.13 识别十六进制无符号整数的状态转换图

识别不同进制数的状态图

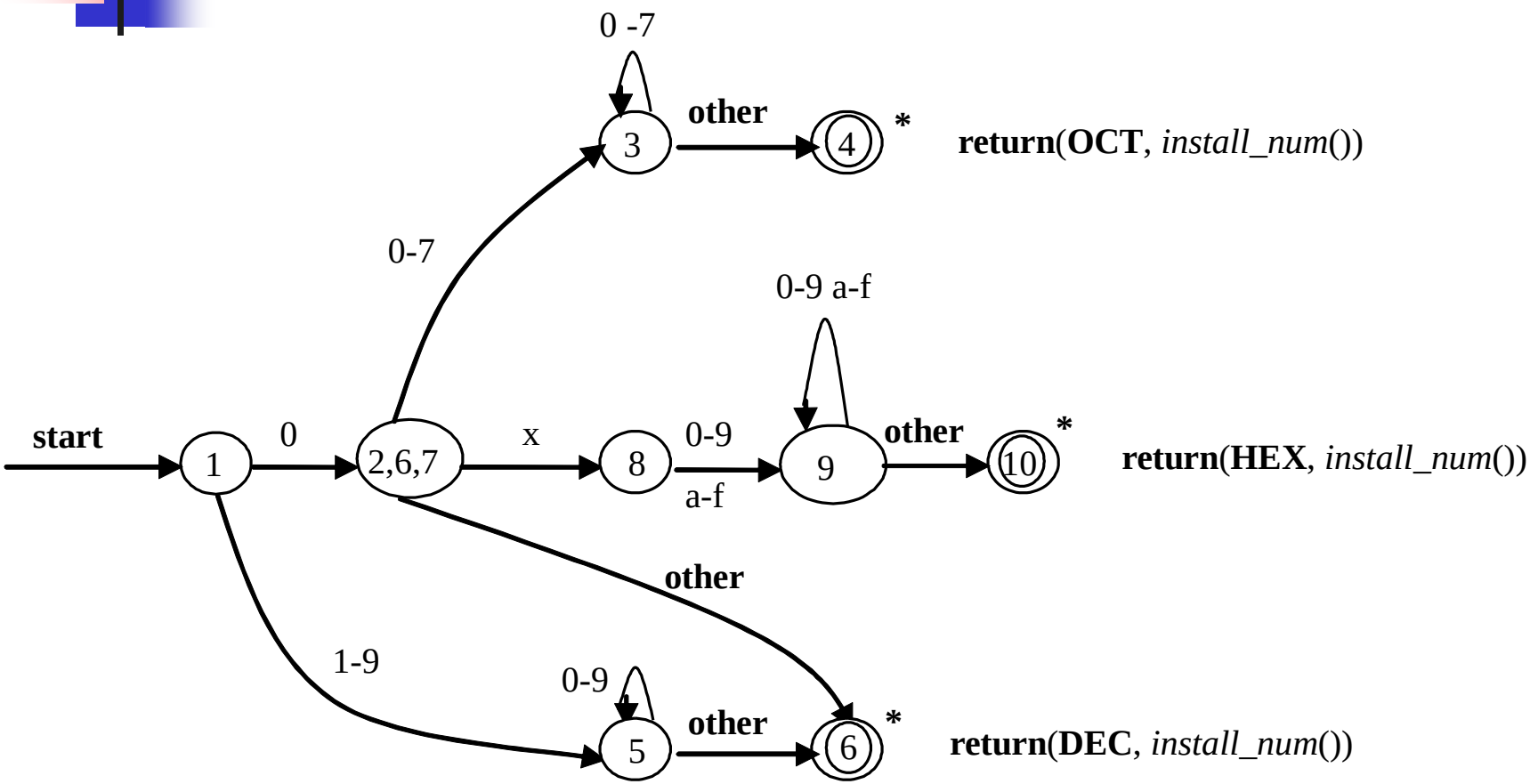
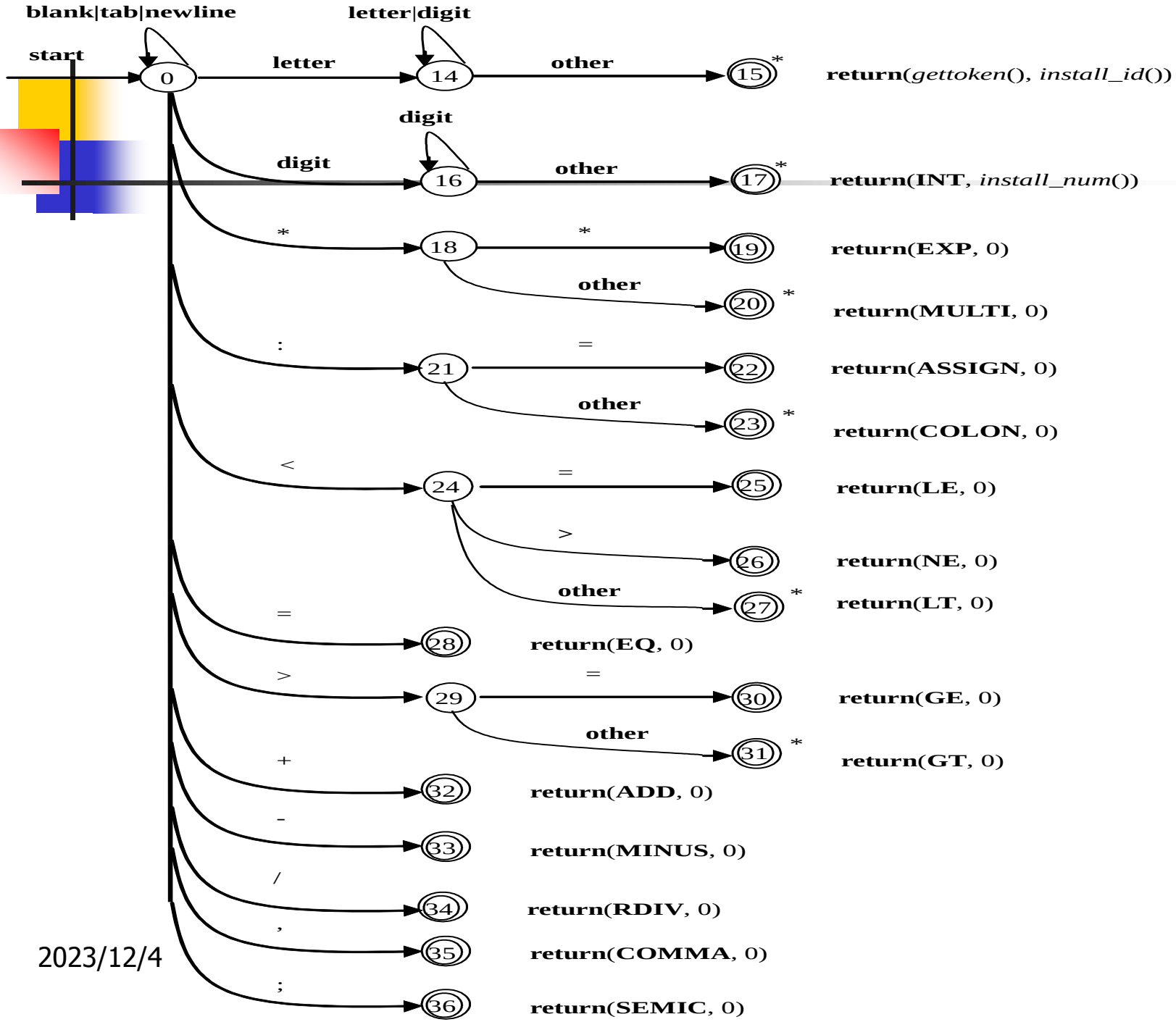


图3.14 识别C语言不同进制无符号整数的状态转换图

2 单词识别的状态转换图表示

- <id> → letter <id_left>
- <id_left> →ε| letter <id_left> | digit <id_left>
- <int> → digit <int_left>
- <int_left> →ε| digit <int_left>
- <assignment> → :=
- <relop> → < | <= | = | < > | > | >=
- <op> → + | - | * | / | **
- <delimiter> → : | , | ;


blank|tab|newline
letter|digit
start
letter
other
return(gettoken(), install_id())
digit
digit
other
return(INT, install_num())
*
*
return(EXP, 0)
other
return(MULTI, 0)
:
=
return(ASSIGN, 0)
other
return(COLON, 0)
<
=
return(LE, 0)
>
return(NE, 0)
other
return(LT, 0)
=
return(EQ, 0)
>
=
return(GE, 0)
other
return(GT, 0)
+
return(ADD, 0)
-
return(MINUS, 0)
/
return(RDIV, 0)
,
return(COMMA, 0)
;
return(SEMIC, 0)

利用状态转换图识别单词

(1) 从初始状态出发；

(2) 读入一个字符；

(3) 按当前字符转入下一状态；

(4) 重复 (2)-(3) 直到无法继续转移为止。

在遇到读入的字符是单词的分界符时，若当前状态是终止状态，说明读入的字符组成了一个单词；否则，说明输入字符串*w*不符合词法规则。

利用状态转换图识别单词

- 如果从状态转换图的初始状态出发，分别沿着所有可能的路径到达终止状态，并将每条路径上的标记依次连接成字符串，则可以得到状态转换图能够识别的所有单词，这些单词组成的集合也就是状态转换图识别的语言。
- 读入字符a时从状态A转换到状态B正好对应着一步推导过程，即$A \Rightarrow aB$，边正好与产生式$(A \rightarrow aB)$相对应

由正则文法构造状态转换图

(1) 以每个语法变量(或其编号)为状态结点，开始符号对应初始状态S；

(2) 增设一个终止状态 T；

(3) 对G中每个形如$A \to aB$的产生式，从状态A到状态B画一条有向弧，并标记为a；

(4) 对G中每个形如$A \to a$的产生式，从状态A到终止状态T画一条标记为a的有向弧；

(5) 对G中每个形如$A \to \varepsilon$的产生式，从状态A到终止状态T画一条标记为any的有向弧，any表示T中的任何符号。

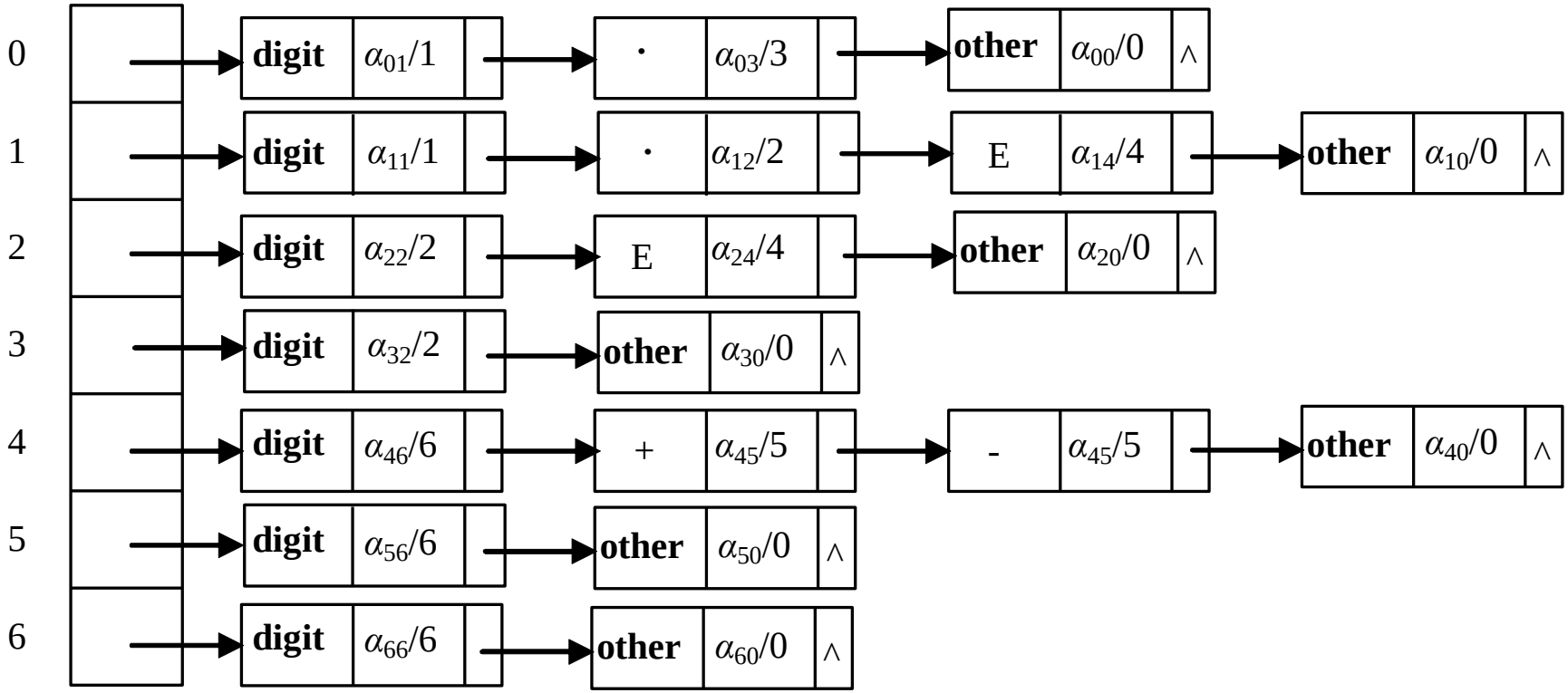
3 几种典型的单词识别问题

- 标识符的识别
- 关键字的识别
- 常数的识别
- 算符和分界符的识别
- 回退

4 状态转换图的实现

- 如果将状态转换图看成是单词的识别规则库的话，则单词识别程序从当前状态(最初为初始状态)出发，读入一个输入字符后，将首先查询该规则库。
- 重复以下过程，直至到达某个终止状态。
  - 如果从当前状态出发有一条边上标记了刚刚读入的输入字符，则单词识别程序将转入这条边所指向的那个状态，并再读入一个输入字符；
  - 否则调用出错处理程序；
  - 将从初始状态到该终止状态所经历的路径上的字符所组成的字符串作为一个单词输出；
  - 并将当前状态重新置为开始状态，以便进行下一个单词的识别；
  - 如果读完输入字符流后仍未进入某个终止状态则调用出错处理程序。

3.3.4 状态转换图的实现



5 词法分析程序的编写

- 状态转移图——教材P93图3.15
- 状态转移图的实现——教材P105图3.22
- 词法分析程序*token_scan*( )
  - 输入：字符流
  - 输出：
    - *symbol*:单词种别
    - *attr*:属性（全局变量）

数据结构与子例程

- 数据结构
  - *ch* 字符变量，存放当前读入的输入字符
  - *token* 字符串变量，存放构成单词的字符串
  - *symbol* 单词种别（词法分析子程序的返回值）
  - *attr* 属性（全局变量）
- 子例程
  - ***install_id*(*token*):**将*token*存入符号表，返回入口指针
  - ***getchar*():**从输入缓冲区中读入一个字符放入*ch*
  - ***retract*():**将向前指针回退一个字符，将*ch*置为空白符
  - ***copytoken*():**返回输入缓冲区中从开始指针*lexeme_beginning*到向前指针*forward*之间的字符串
  - ***isletter*() *isalpha*() *isalnum*()**

图　的状态转换图的实现算法

```
token token_scan()
{ char ch;
char* token;
ch = getchar();
while (ch = =blank || ch = = tab || ch = = newline) {
ch = getchar();
    lexeme_beginning++;
}
if (isalpha(ch)) {ch = getchar();
    while (isalnum(ch))
ch = getchar();
retract(1);
token = copytoken();
return(gettoken(token), install_id(token));}
```

```
else
if (isdigit(ch)) {
    ch = getchar();
    while (isdigit(ch))
      ch = getchar();
retract(1);
token = copytoken();
return(INT, install_num(token));
}
else
switch(ch)
{
    case '*': ch = getchar();
              if(ch = = '*') return(EXP, 0);
              else {
                  retract(1);
                  return(MULTI, 0);
              } break;
```

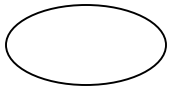
```
case ':': ch = getchar();
            if(ch = = '=') return(ASSIGN, 0);
            else { retract(1); return(COLON, 0);
            } break;
case '<': ch = getchar();
            if(ch = = '=') return(LE, 0);
            else if(ch = = '>') return(NE, 0);
else { retract(1);   return(LT, 0);
            } break;
case '=': return(EQ, 0); break;
case '>': ch = getchar();
            if(ch = = '=') return(GE, 0);
            else { retract(1); return(GT, 0);
            } break;
case '+': return(PLUS, 0); break;
case '-': return(MINUS, 0); break;
case '/': return(RDIV, 0); break;
case ',': return(COMMA, 0); break;
case ';': return(SEMIC, 0); break;
default: error_handle(); break;}return;}
```

需要说明的问题

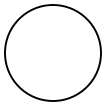
- 缓冲区预处理，超前搜索
- 关键字的处理，符号表的实现
- ***install_id*()**：查找效率，算法的优化实现
- 词法错误处理
- 由于高级语言的词组成的集合为3型语言，所以，这里讨论的词法分析技术可以用于处理所有的3型语言，也就是所有的可以用3型文法描述的语言。如：信息检索系统的查询语言、命令语言等

语法图

内的文字表示语法成分（短语）

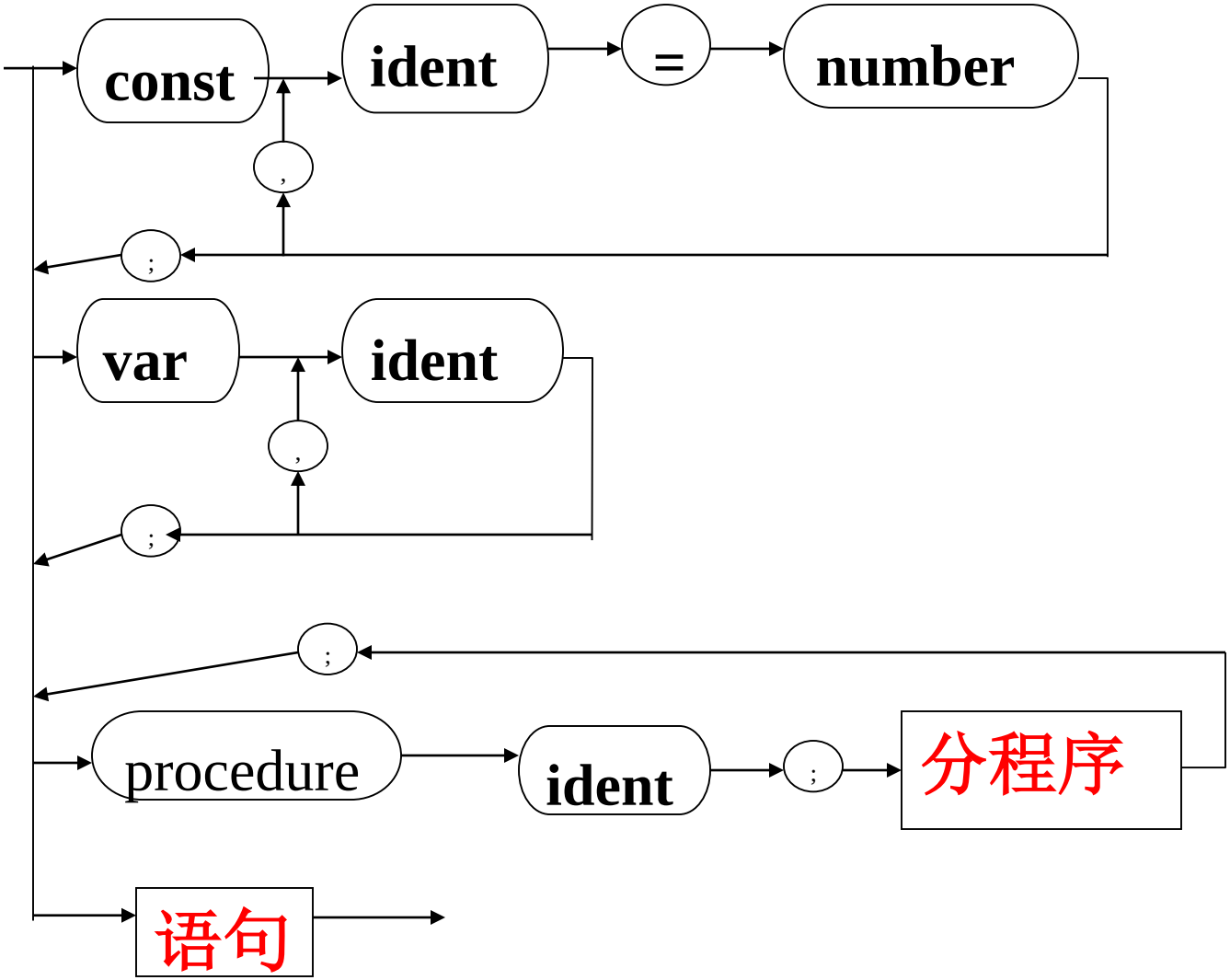


或



内的文字表示单词符号

分程序



PL/0语言的EBNF表示

构成EBNF的元素—（非终结符，终结符，开始符，规则）

EBNF 的元符号：

< > 用左右尖括号括起来的内容为非终结符

::= 读做‘定义为’ ::=的左部由右部定义

→ 读做‘定义为’ → 的左部由右部定义

| 读做‘或’ 表示右部候选内容

{ } 表示花括号内的内容可重复任意次或限定次数

[ ] 表示方括号内的内容为任选项

( ) 表示圆括号内的内容优先

例：用EBNF描述<整数>的定义 ：

<整数>::=[+|-]<数字>{<数字>}

<数字>::=0|1|2|3|4|5|6|7|8|9

或

<整数>::=[+|-]<非零数字>{<数字>}|0

<非零数字>::=1|2|3|4|5|6|7|8|9

<数字>::=0|<非零数字>

PL/0语言是PASCAL语言的子集

同PASCAL

作用域规则（内层可引用包围它的外层定义的标识符），上下文约束， 过程可嵌套定义，可递归调用

子集

- 数据类型,只有整型
- 数据结构 ,只有简变和常数
- 数字最多为14位
- 标识符的有效长度是10
- 语句种类
- 过程无参，最多可嵌套三层

目标代码类p-code

目标代码类p-code是一种栈式机的汇编语言。

栈式机系统结构:没有累加器和寄存器，只有存储栈指针 .

所有运算都在栈顶（零地址机）

指令格式:

f	l	a

f　　功能码

l　　层次差　（标识符引用层减去定义层）

a　　根据不同的指令有所区别

指令功能表

LIT 0 a	将常数值取到栈顶，a为常数值
LOD l a	将变量值取到栈顶，a为偏移量，l为层差
STO l a	将栈顶内容送入某变量单元中，a为偏移量，l为层差
CAL l a	调用过程，a为过程地址，l为层差
INT 0 a	在运行栈中为被调用的过程开辟a个单元的数据区
JMP 0 a	无条件跳转至a地址
JPC 0 a	条件跳转，当栈顶布尔值非真则跳转至a地址，否则顺序执行
OPR 0 0	过程调用结束后,返回调用点并退栈
OPR 0 1	栈顶元素取反
OPR 0 2	次栈顶与栈顶相加，退两个栈元素，结果值进栈
OPR 0 3	次栈顶减去栈顶，退两个栈元素，结果值进栈
OPR 0 4	次栈顶乘以栈顶，退两个栈元素，结果值进栈
OPR 0 5	次栈顶除以栈顶，退两个栈元素，结果值进栈
OPR 0 6	栈顶元素的奇偶判断，结果值在栈顶
OPR 0 7	
OPR 0 8	次栈顶与栈顶是否相等，退两个栈元素，结果值进栈
OPR 0 9	次栈顶与栈顶是否不等，退两个栈元素，结果值进栈
OPR 0 10	次栈顶是否小于栈顶，退两个栈元素，结果值进栈
OPR 0 11	次栈顶是否大于等于栈顶，退两个栈元素，结果值进栈
OPR 0 12	次栈顶是否大于栈顶，退两个栈元素，结果值进栈
OPR 0 13	次栈顶是否小于等于栈顶，退两个栈元素，结果值进栈
OPR 0 14	栈顶值输出至屏幕
OPR 0 15	屏幕输出换行
OPR 0 16	从命令行读入一个输入置于栈顶

```
const   a=10;
var     b,c;
procedure p;
   begin
      c:=b+a;
   end;
begin
   read(b);
   while b#0 do
   begin
      call p;
      write(2*c);
      read(b);
   end
end.
```

```
( 0) jmp 0 8   转向主程序入口
( 1) jmp 0 2   转向过程p入口
( 2) int 0 3   过程p入口,为过程p开辟空间
( 3) lod 1 3   取变量b的值到栈顶
( 4) lit 0 10  取常数10到栈顶
( 5) opr 0 2   次栈顶与栈顶相加
( 6) sto 1 4   栈顶值送变量c中
( 7) opr 0 0   退栈并返回调用点(16)
( 8) int 0 5   主程序入口开辟5个栈空间
( 9) opr 0 16  从命令行读入值置于栈顶
(10) sto 0 3   将栈顶值存入变量b中
(11) lod 0 3   将变量b的值取至栈顶
(12) lit 0 0   将常数值0进栈
(13) opr 0 9   次栈顶与栈顶是否不等
(14) jpc 0 24  等时转(24)（条件不满足转）
(15) cal 0 2   调用过程p
(16) lit 0 2   常数值2进栈
(17) lod 0 4   将变量c的值取至栈顶
(18) opr 0 4   次栈顶与栈顶相乘(2*c)
(19) opr 0 14  栈顶值输出至屏幕
(20) opr 0 15  换行
(21) opr 0 16  从命令行读取值到栈顶
(22) sto 0 3   栈顶值送变量b中
(23) jmp 0 11  无条件转到循环入口(11)
(24) opr 0 0   结束退栈
```

词法分析过程GETSYM所要完成的任务：

- 从源程序读字符（getch）
- 滤空格
- 识别保留字
- 识别标识符
- 拼数
- 识别单字符单词
- 拼双字符单词

- 词法分析过程:GETSYM框图（见教材图2.5）
- 程序（ procedure getsym）

当识别到标识符时先查保留字表

- 保留字表：（ begin （* main * ） ）

word[1]:=‘begin ‘; word[2]:=‘call ‘;

...

word[13]:=‘write ‘;

查到时找到相应的内部表示

Wsym[1]:=beginsym; wsym[2]:=callsym;

...

wsym[13]:=writesym;

- 字符对应的单词表：

ssym['+']:=plus; ssym['-']:=minus;

…

ssym[';']:=semicolon;

- 词法分析如何把单词传递给语法分析

```
type
  symbol=(nul, ident, number, plus, …, varsym, procsym);
```

3个全程量

SYM:symbol;

ID:alfa;

NUM:integer;

通过三个全程量 SYM 、ID和NUM 将识别出的单词信息传递给语法分析程序。

- SYM：存放单词的类别

  如：有程序段落为：

  begin initial := 60；end

  对应单词翻译后变为：

  begin ⟹ beginsym，initial ⟹ ident，

  ':= ' ⟹ becomes，60 ⟹ number，'；' ⟹ semicolon，

  end ⟹ endsym 。

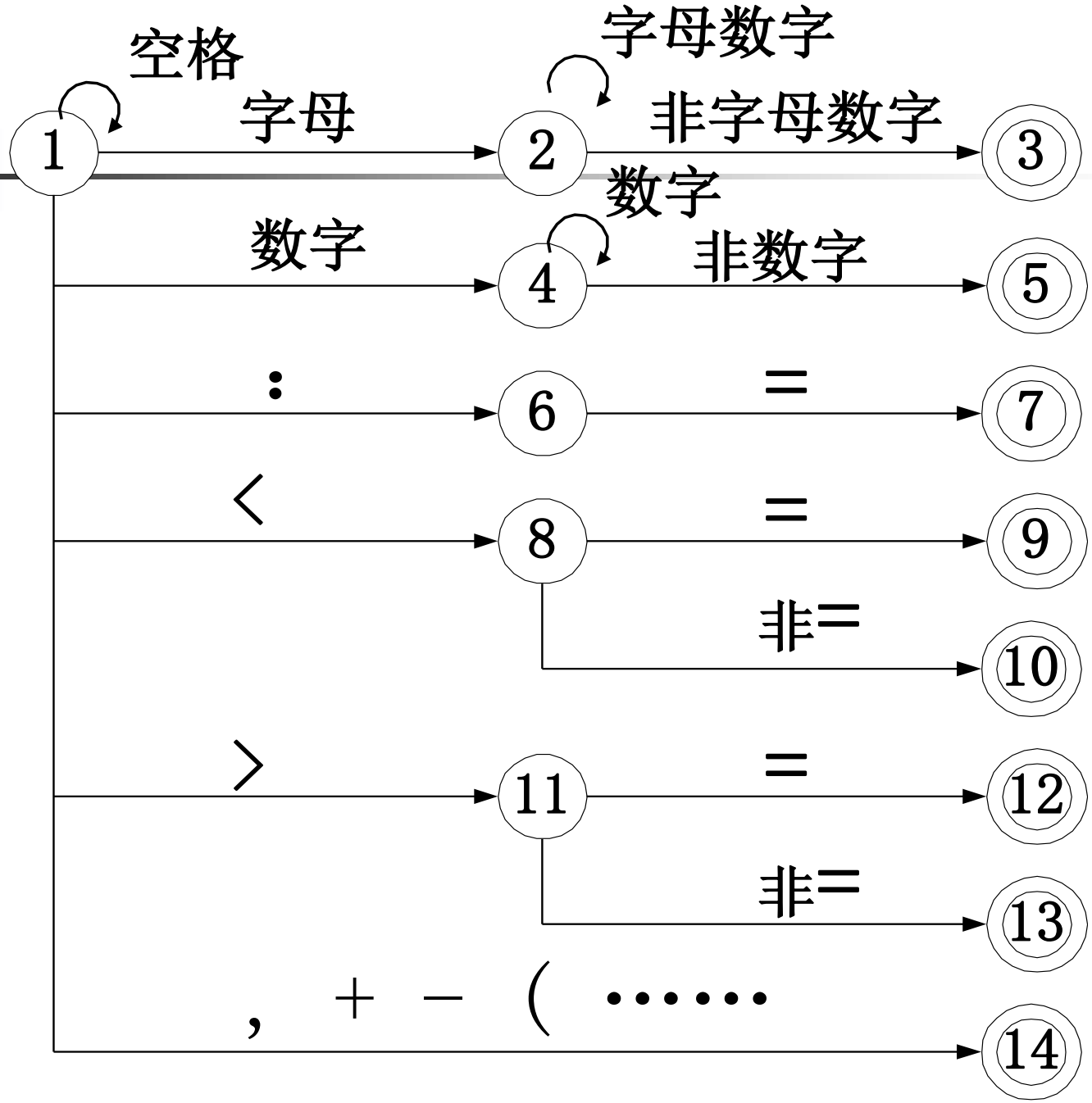
- ID： 存放用户所定义的标识符的值 如： initial （在SYM中放ident，在ID中放initial）
- NUM：存放用户定义的数 如：60
- （在SYM中放number,在NUM中放60）

使用状态转换图实现词法分析程序的设计方法

词法分析程序的设计---使用状态转换图实现

○ 表示状态，对应每个状态编一段程序，每个状态调用取字符程序，根据当前字符转到不同的状态，并做相应操作。

◎ 表示终态，已识别出一个单词。


空格
字母
字母数字
非字母数字
数字
数字
非数字
:
=
<
=
非=
>
=
非=
, + − (……
1
2
3
4
5
6
7
8
9
10
11
12
13
14